Bibliotecas

Em [1]:
```
from __future__ import annotations

import sys
import os
import csv
import random
import tempfile
from pathlib import Path
from collections import Counter

import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

import torch
import torch.nn as nn
import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader
from torchvision import transforms as T
from torchvision.utils import make_grid, save_image

from torchmetrics.image.fid import FrechetInceptionDistance
from torchmetrics.image.kid import KernelInceptionDistance
```

REPRODUTIBILIDADE

Em [2]:
```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

print("=" * 60)
print("Reprodutibilidade configurada")
print(f"Seed base: {SEED}")
print("=" * 60)
```

```
============================================================
Reprodutibilidade configurada
Seed base: 42
============================================================
```

PATHS DO PROJETO

Em [3]:
```
PROJECT_ROOT = Path("..").resolve()
SCRIPTS_DIR = PROJECT_ROOT / "scripts"
```

```python
KAGGLE_ROOT = PROJECT_ROOT / "ArtBench-10"

print("PROJECT_ROOT =", PROJECT_ROOT)
print("SCRIPTS_DIR  =", SCRIPTS_DIR)
print("KAGGLE_ROOT  =", KAGGLE_ROOT)

if not KAGGLE_ROOT.exists():
    raise FileNotFoundError(f"Não encontrei a pasta do dataset: {KAGGLE_ROOT}")

if not (SCRIPTS_DIR / "artbench_local_dataset.py").exists():
    raise FileNotFoundError(
        f"Não encontrei o ficheiro artbench_local_dataset.py em {SCRIPTS_DIR}"
    )

if str(SCRIPTS_DIR) not in sys.path:
    sys.path.insert(0, str(SCRIPTS_DIR))

print("Paths configurados com sucesso.")
```

```
PROJECT_ROOT = C:\Users\joaoa\Desktop\Lara\IAG
SCRIPTS_DIR  = C:\Users\joaoa\Desktop\Lara\IAG\scripts
KAGGLE_ROOT  = C:\Users\joaoa\Desktop\Lara\IAG\ArtBench-10
Paths configurados com sucesso.
```

CARREGAR O DATASET

Em [4]:

```python
from artbench_local_dataset import load_kaggle_artbench10_splits

print("=" * 60)
print("A carregar o ArtBench-10...")
print("=" * 60)

hf_ds = load_kaggle_artbench10_splits(KAGGLE_ROOT)

train_hf = hf_ds["train"]
test_hf = hf_ds["test"]

print("Train size:", len(train_hf))
print("Test size :", len(test_hf))
print("Columns   :", train_hf.column_names)

label_feature = train_hf.features["label"]
class_names = list(label_feature.names)
num_classes = len(class_names)

print("Número de classes:", num_classes)
print("Classes:", class_names)
```

```
c:\Users\joaoa\Desktop\Lara\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See https://ipywidget
s.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
============================================================
A carregar o ArtBench-10...
============================================================
Dataset source: kaggle root='C:\Users\joaoa\Desktop\Lara\IAG\ArtBench-10'
Train size: 50000
Test size : 10000
Columns   : ['image', 'label']
Número de classes: 10
Classes: ['impressionism', 'realism', 'romanticism', 'expressionism', 'baroque',
'post_impressionism', 'art_nouveau', 'surrealism', 'ukiyo_e', 'renaissance']
```

DISTRIBUIÇÃO DAS CLASSES

In [5]:
```python
train_counts = Counter(train_hf["label"])

print("=" * 60)
print("Distribuição das classes no treino")
print("=" * 60)

for cid, name in enumerate(class_names):
    print(f"{cid:2d} | {name:>15s} | {train_counts.get(cid, 0):6d}")
```

```
============================================================
Distribuição das classes no treino
============================================================
 0 |   impressionism |   5000
 1 |         realism |   5000
 2 |     romanticism |   5000
 3 |   expressionism |   5000
 4 |         baroque |   5000
 5 | post_impressionism |   5000
 6 |     art_nouveau |   5000
 7 |      surrealism |   5000
 8 |         ukiyo_e |   5000
 9 |     renaissance |   5000
```

CONFIGURAÇÃO DE IMAGEM E DATALOADER

In [6]:
```python
IMAGE_SIZE = 32
BATCH_SIZE = 64
NUM_WORKERS = 2
TRAIN_FRACTION = 1.0

def resolve_num_workers(requested: int) -> int:
    if "ipykernel" in sys.modules and int(requested) > 0:
        print("Notebook detetado: num_workers será forçado para 0.")
        return 0
    return int(requested)

EFFECTIVE_NUM_WORKERS = resolve_num_workers(NUM_WORKERS)

transform = T.Compose([
    T.Resize(IMAGE_SIZE, interpolation=T.InterpolationMode.BILINEAR),
    T.CenterCrop(IMAGE_SIZE),
    T.ToTensor()
```

```
])

print("=" * 60)
print("Configuração dos dados")
print(f"IMAGE_SIZE = {IMAGE_SIZE}")
print(f"BATCH_SIZE = {BATCH_SIZE}")
print(f"NUM_WORKERS pedido = {NUM_WORKERS}")
print(f"NUM_WORKERS efetivo = {EFFECTIVE_NUM_WORKERS}")
print("=" * 60)


class ArtBenchTorchDataset(Dataset):
    def __init__(self, hf_split, transform=None, indices=None):
        self.ds = hf_split
        self.transform = transform
        self.indices = list(range(len(hf_split))) if indices is None else list(i

    def __len__(self):
        return len(self.indices)

    def __getitem__(self, idx):
        original_idx = self.indices[idx]
        sample = self.ds[original_idx]
        image = sample["image"]
        label = int(sample["label"])
        x = self.transform(image) if self.transform else image
        return x, label, original_idx


def build_fraction_indices(n_total: int, fraction: float, seed: int = 42):
    n_keep = max(1, int(round(n_total * fraction)))
    rng = np.random.RandomState(seed)
    idx = np.arange(n_total)
    rng.shuffle(idx)
    return idx[:n_keep].tolist()


train_indices = build_fraction_indices(len(train_hf), TRAIN_FRACTION, seed=SEED)

train_ds = ArtBenchTorchDataset(train_hf, transform=transform, indices=train_ind
test_ds = ArtBenchTorchDataset(test_hf, transform=transform)

train_loader = DataLoader(
    train_ds,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=EFFECTIVE_NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

test_loader = DataLoader(
    test_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=EFFECTIVE_NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
)

print("Train dataset length:", len(train_ds))
print("Test dataset length :", len(test_ds))
```

```
print("Train batches       :", len(train_loader))
print("Test batches        :", len(test_loader))
```

```
Notebook detetado: num_workers será forçado para 0.
============================================================
Configuração dos dados
IMAGE_SIZE = 32
BATCH_SIZE = 64
NUM_WORKERS pedido = 2
NUM_WORKERS efetivo = 0
============================================================
Train dataset length: 50000
Test dataset length : 10000
Train batches       : 782
Test batches        : 157
```

SUBSET DE TREINO A PARTIR DO CSV

In [7]:

```
TRAINING_CSV_PATH = Path("training_20_percent.csv")
INDEX_COLUMN = "train_id_original"

def load_ids_from_training_csv(csv_path: Path, index_column: str = "train_id_ori
    csv_path = Path(csv_path)

    if not csv_path.exists():
        raise FileNotFoundError(
            f"Não encontrei o CSV do subset: {csv_path}"
        )

    ids = []
    with open(csv_path, "r", encoding="utf-8", newline="") as f:
        reader = csv.DictReader(f)

        if index_column not in (reader.fieldnames or []):
            raise ValueError(
                f"A coluna {index_column!r} não existe. Colunas disponíveis: {re
            )

        for row in reader:
            value = str(row.get(index_column, "")).strip()
            if value != "":
                ids.append(int(value))

    if len(ids) == 0:
        raise ValueError("O CSV foi lido, mas não continha IDs válidos.")

    return ids


subset_ids = load_ids_from_training_csv(TRAINING_CSV_PATH, index_column=INDEX_CO

subset_train_ds = ArtBenchTorchDataset(train_hf, transform=transform, indices=su
subset_train_loader = DataLoader(
    subset_train_ds,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=EFFECTIVE_NUM_WORKERS,
    pin_memory=torch.cuda.is_available()
```

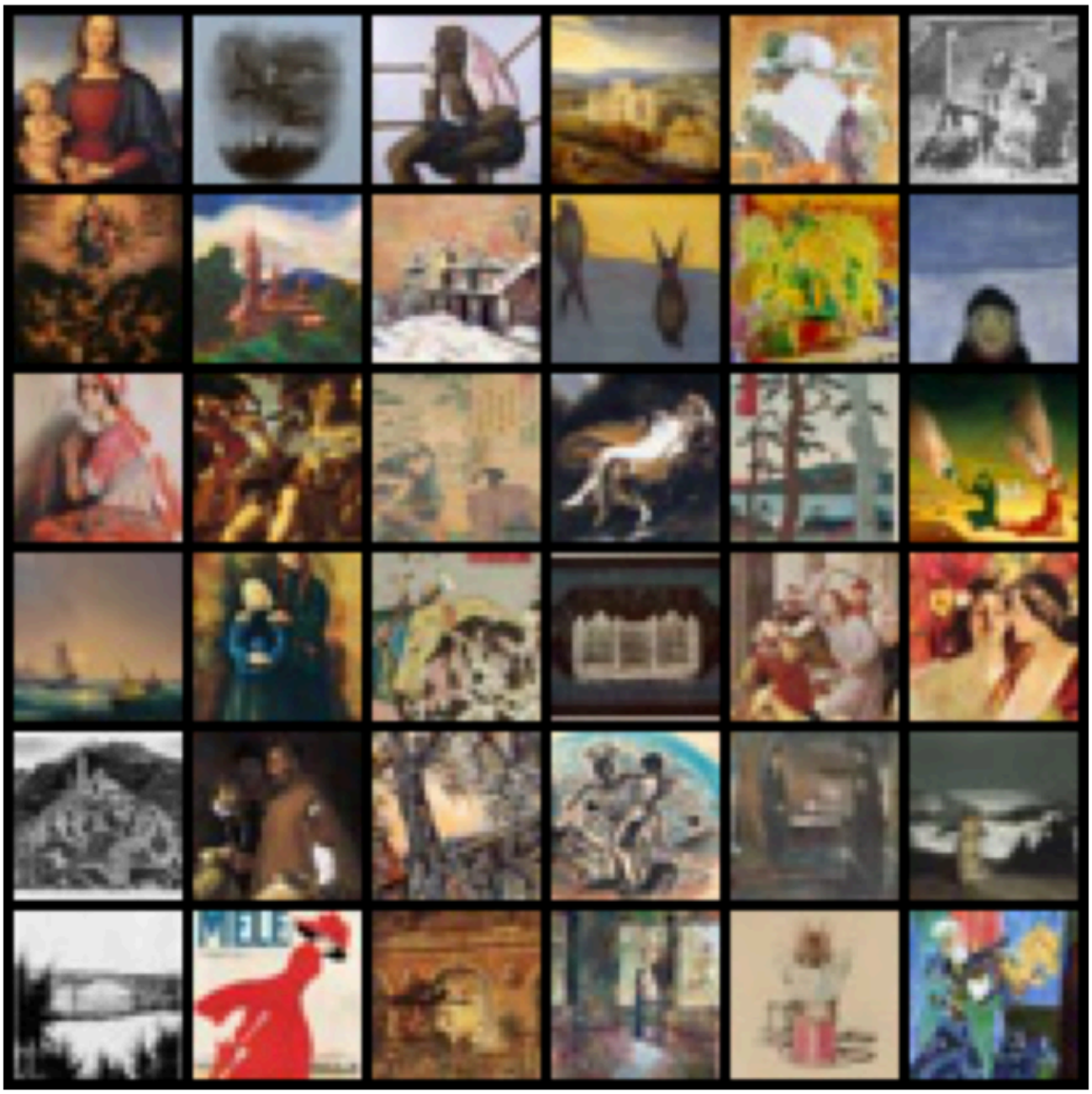
```
)

print("=" * 60)
print("Subset de treino carregado")
print(f"Número de IDs no CSV: {len(subset_ids)}")
print(f"Tamanho do dataset subset: {len(subset_train_ds)}")
print(f"Número de batches subset : {len(subset_train_loader)}")
print("=" * 60)
```

```
============================================================
Subset de treino carregado
Número de IDs no CSV: 10000
Tamanho do dataset subset: 10000
Número de batches subset : 157
============================================================
```

VISUALIZAÇÃO DE EXEMPLOS

In [8]:
```
def show_batch_grid(loader, class_names, n_images=36, nrow=6, title="Sample Grid
    x, y, idx = next(iter(loader))
    x = x[:n_images]
    y = y[:n_images]

    grid = make_grid(x, nrow=nrow, padding=2)
    np_img = grid.permute(1, 2, 0).cpu().numpy()

    plt.figure(figsize=(8, 8))
    plt.imshow(np_img)
    plt.axis("off")
    plt.title(title)
    plt.show()

    labels_str = [class_names[int(v)] for v in y]
    print("Labels:", labels_str)

show_batch_grid(subset_train_loader, class_names, n_images=36, nrow=6, title="Ar
```

ArtBench-10 subset samples



```
Labels: ['renaissance', 'romanticism', 'expressionism', 'romanticism', 'post_impr
essionism', 'realism', 'renaissance', 'post_impressionism', 'post_impressionism',
'surrealism', 'post_impressionism', 'expressionism', 'art_nouveau', 'baroque', 'u
kiyo_e', 'romanticism', 'ukiyo_e', 'surrealism', 'romanticism', 'post_impressioni
sm', 'ukiyo_e', 'surrealism', 'renaissance', 'art_nouveau', 'realism', 'baroque',
'expressionism', 'surrealism', 'expressionism', 'realism', 'realism', 'art_nouvea
u', 'impressionism', 'expressionism', 'art_nouveau', 'post_impressionism']
```

In [9]:

```
# ========================
# MODELO VAE
# ========================

class SimpleVAE(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.latent_dim = latent_dim

        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, 4, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 4, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten()
        )
```

```python
        self.fc_mu = nn.Linear(64 * 8 * 8, latent_dim)
        self.fc_logvar = nn.Linear(64 * 8 * 8, latent_dim)

        self.decoder_input = nn.Linear(latent_dim, 64 * 8 * 8)

        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 4, 2, 1),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 4, 2, 1),
            nn.Sigmoid()
        )

    def encode(self, x):
        h = self.encoder(x)
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h = self.decoder_input(z).view(-1, 64, 8, 8)
        return self.decoder(h)

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        return recon, mu, logvar


# ========================
# DCGAN
# ========================

class DCGANGenerator(nn.Module):
    def __init__(self, noise_dim=100):
        super().__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(noise_dim, 256, 4, 1, 0, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(True),

            nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),

            nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),

            nn.ConvTranspose2d(64, 3, 4, 2, 1, bias=False),
            nn.Tanh()
        )

    def forward(self, z):
```

```python
        return self.main(z)


class DCGANDiscriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, 3, 2, 1),
            nn.LeakyReLU(0.2),

            nn.Conv2d(64, 128, 3, 2, 1),
            nn.LeakyReLU(0.2),

            nn.Conv2d(128, 256, 3, 2, 1),
            nn.LeakyReLU(0.2)
        )

        self.classifier = nn.Linear(256 * 4 * 4, 1)

    def forward(self, x):
        h = self.features(x)
        h = h.view(h.size(0), -1)
        return self.classifier(h).squeeze(1)


# =========================
# TOY DIFFUSION U-NET
# =========================

class TinyUNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.down = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1),
            nn.ReLU()
        )
        self.up = nn.Sequential(
            nn.Conv2d(64, 3, 3, padding=1)
        )

    def forward(self, x, t):
        return self.up(self.down(x))
```

In [10]:

```python
# =========================
# INICIALIZAÇÃO
# =========================

LATENT_DIM = 128
NOISE_DIM = 100
EPOCHS = 10
LR_VAE = 1e-3
LR_GAN = 2e-4
LR_DIFF = 1e-3

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

vae_model = SimpleVAE(latent_dim=LATENT_DIM).to(device)
gan_generator = DCGANGenerator(noise_dim=NOISE_DIM).to(device)
gan_discriminator = DCGANDiscriminator().to(device)
diffusion_model = TinyUNet().to(device)
```

```
print("=" * 60)
print("Modelos instanciados")
print(f"Device: {device}")
print("=" * 60)

vae_optimizer = torch.optim.Adam(vae_model.parameters(), lr=LR_VAE)
gan_g_optimizer = torch.optim.Adam(gan_generator.parameters(), lr=LR_GAN, betas=
gan_d_optimizer = torch.optim.Adam(gan_discriminator.parameters(), lr=LR_GAN, be
diff_optimizer = torch.optim.Adam(diffusion_model.parameters(), lr=LR_DIFF)

gan_criterion = nn.BCEWithLogitsLoss()

T_STEPS = 100
betas = torch.linspace(1e-4, 0.02, T_STEPS, device=device)
alphas = 1.0 - betas
alphas_cumprod = torch.cumprod(alphas, dim=0)
```

```
============================================================
Modelos instanciados
Device: cpu
============================================================
```

In [11]:

```
def vae_loss_function(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(recon_x, x, reduction="sum")
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div


def to_01(x):
    x = x.detach().cpu()
    if x.min() < 0:
        x = (x + 1.0) / 2.0
    return x.clamp(0, 1)


def to_uint8(x):
    x = to_01(x)
    return (x * 255.0).round().to(torch.uint8)
```

In [12]:

```
def train_vae(model, loader, optimizer, epochs, device):
    history = []

    print("=" * 60)
    print("Treino do VAE")
    print("=" * 60)

    model.train()

    for epoch in range(epochs):
        running_loss = 0.0

        for batch_idx, (real_imgs, labels, indices) in enumerate(loader):
            real_imgs = real_imgs.to(device)

            optimizer.zero_grad()
            recon_imgs, mu, logvar = model(real_imgs)
            loss = vae_loss_function(recon_imgs, real_imgs, mu, logvar)
            loss.backward()
            optimizer.step()
```

```
            running_loss += loss.item()

        epoch_loss = running_loss / len(loader.dataset)
        history.append(epoch_loss)

        print(f"[VAE] Epoch {epoch + 1}/{epochs} | loss = {epoch_loss:.6f}")

    return history
```

In [13]:
```
def train_vae(model, loader, optimizer, epochs, device):
    history = []

    print("=" * 60)
    print("Treino do VAE")
    print("=" * 60)

    model.train()

    for epoch in range(epochs):
        running_loss = 0.0

        for batch_idx, (real_imgs, labels, indices) in enumerate(loader):
            real_imgs = real_imgs.to(device)

            optimizer.zero_grad()
            recon_imgs, mu, logvar = model(real_imgs)
            loss = vae_loss_function(recon_imgs, real_imgs, mu, logvar)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        epoch_loss = running_loss / len(loader.dataset)
        history.append(epoch_loss)

        print(f"[VAE] Epoch {epoch + 1}/{epochs} | loss = {epoch_loss:.6f}")

    return history
```

In [14]:
```
def train_gan(generator, discriminator, loader, opt_g, opt_d, criterion, epochs,
    g_history = []
    d_history = []

    print("=" * 60)
    print("Treino da DCGAN")
    print("=" * 60)

    generator.train()
    discriminator.train()

    for epoch in range(epochs):
        epoch_g_loss = 0.0
        epoch_d_loss = 0.0

        for batch_idx, (real_imgs, labels, indices) in enumerate(loader):
            real_imgs = real_imgs.to(device)
            batch_size = real_imgs.size(0)
```

```
            real_targets = torch.ones(batch_size, device=device)
            fake_targets = torch.zeros(batch_size, device=device)

            # ---------
            # Train D
            # ---------
            opt_d.zero_grad()

            real_logits = discriminator(real_imgs)
            d_loss_real = criterion(real_logits, real_targets)

            noise = torch.randn(batch_size, noise_dim, 1, 1, device=device)
            fake_imgs = generator(noise)
            fake_logits = discriminator(fake_imgs.detach())
            d_loss_fake = criterion(fake_logits, fake_targets)

            d_loss = d_loss_real + d_loss_fake
            d_loss.backward()
            opt_d.step()

            # ---------
            # Train G
            # ---------
            opt_g.zero_grad()

            noise = torch.randn(batch_size, noise_dim, 1, 1, device=device)
            fake_imgs = generator(noise)
            fake_logits = discriminator(fake_imgs)

            g_loss = criterion(fake_logits, real_targets)
            g_loss.backward()
            opt_g.step()

            epoch_d_loss += d_loss.item()
            epoch_g_loss += g_loss.item()

        avg_d = epoch_d_loss / len(loader)
        avg_g = epoch_g_loss / len(loader)

        d_history.append(avg_d)
        g_history.append(avg_g)

        print(f"[GAN] Epoch {epoch + 1}/{epochs} | D loss = {avg_d:.6f} | G loss

    return g_history, d_history
```

In [15]:

```
def train_diffusion(model, loader, optimizer, epochs, device, betas, alphas_cump
    history = []

    print("=" * 60)
    print("Treino do modelo de difusão")
    print("=" * 60)

    model.train()
    n_steps = betas.size(0)

    for epoch in range(epochs):
        running_loss = 0.0

        for batch_idx, (real_imgs, labels, indices) in enumerate(loader):
```

```
            real_imgs = real_imgs.to(device)
            batch_size = real_imgs.size(0)

            t = torch.randint(0, n_steps, (batch_size,), device=device)
            noise = torch.randn_like(real_imgs)

            alpha_bar_t = alphas_cumprod[t].view(-1, 1, 1, 1)
            noisy_imgs = torch.sqrt(alpha_bar_t) * real_imgs + torch.sqrt(1 - al

            pred_noise = model(noisy_imgs, t)
            loss = F.mse_loss(pred_noise, noise)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        epoch_loss = running_loss / len(loader)
        history.append(epoch_loss)

        print(f"[Diffusion] Epoch {epoch + 1}/{epochs} | loss = {epoch_loss:.6f}

    return history
```

In [16]:

```
vae_history = train_vae(
    model=vae_model,
    loader=subset_train_loader,
    optimizer=vae_optimizer,
    epochs=EPOCHS,
    device=device
)

gan_g_history, gan_d_history = train_gan(
    generator=gan_generator,
    discriminator=gan_discriminator,
    loader=subset_train_loader,
    opt_g=gan_g_optimizer,
    opt_d=gan_d_optimizer,
    criterion=gan_criterion,
    epochs=EPOCHS,
    device=device,
    noise_dim=NOISE_DIM
)

diff_history = train_diffusion(
    model=diffusion_model,
    loader=subset_train_loader,
    optimizer=diff_optimizer,
    epochs=EPOCHS,
    device=device,
    betas=betas,
    alphas_cumprod=alphas_cumprod
)
```

```
============================================================
Treino do VAE
============================================================
[VAE] Epoch 1/10 | loss = 122.068575
[VAE] Epoch 2/10 | loss = 91.637557
[VAE] Epoch 3/10 | loss = 80.664032
[VAE] Epoch 4/10 | loss = 75.700176
[VAE] Epoch 5/10 | loss = 73.717453
[VAE] Epoch 6/10 | loss = 72.800862
[VAE] Epoch 7/10 | loss = 71.993802
[VAE] Epoch 8/10 | loss = 71.714209
[VAE] Epoch 9/10 | loss = 71.172223
[VAE] Epoch 10/10 | loss = 71.119673
============================================================
Treino da DCGAN
============================================================
[GAN] Epoch 1/10 | D loss = 1.010853 | G loss = 1.675449
[GAN] Epoch 2/10 | D loss = 1.161470 | G loss = 1.153745
[GAN] Epoch 3/10 | D loss = 1.412265 | G loss = 0.767176
[GAN] Epoch 4/10 | D loss = 1.342653 | G loss = 0.753005
[GAN] Epoch 5/10 | D loss = 1.359063 | G loss = 0.769934
[GAN] Epoch 6/10 | D loss = 1.357885 | G loss = 0.781902
[GAN] Epoch 7/10 | D loss = 1.358137 | G loss = 0.794485
[GAN] Epoch 8/10 | D loss = 1.368497 | G loss = 0.778402
[GAN] Epoch 9/10 | D loss = 1.364626 | G loss = 0.748811
[GAN] Epoch 10/10 | D loss = 1.377752 | G loss = 0.737361
============================================================
Treino do modelo de difusão
============================================================
[Diffusion] Epoch 1/10 | loss = 0.325693
[Diffusion] Epoch 2/10 | loss = 0.218703
[Diffusion] Epoch 3/10 | loss = 0.212393
[Diffusion] Epoch 4/10 | loss = 0.209434
[Diffusion] Epoch 5/10 | loss = 0.205444
[Diffusion] Epoch 6/10 | loss = 0.197697
[Diffusion] Epoch 7/10 | loss = 0.193771
[Diffusion] Epoch 8/10 | loss = 0.192979
[Diffusion] Epoch 9/10 | loss = 0.189476
[Diffusion] Epoch 10/10 | loss = 0.186259
```

In [17]:

```python
def collect_real_images(loader, n_samples=5000):
    print(f"[Real] A recolher {n_samples} imagens reais...")

    images = []
    total = 0

    for batch_idx, batch in enumerate(loader):
        x = batch[0]
        images.append(to_uint8(x))
        total += x.size(0)

        print(f"  Batch {batch_idx + 1}: acumuladas = {total}")

        if total >= n_samples:
            break

    images = torch.cat(images, dim=0)[:n_samples]
    print(f"[Real] Shape final: {images.shape}")
    return images
```

In [18]:
```
@torch.no_grad()
def sample_from_vae(model, n_samples=5000, latent_dim=128, batch_size=256, devic
    print(f"[VAE] A gerar {n_samples} imagens...")
    model.eval()

    outputs = []
    generated = 0

    while generated < n_samples:
        current_bs = min(batch_size, n_samples - generated)
        z = torch.randn(current_bs, latent_dim, device=device)

        x = model.decode(z)
        outputs.append(to_uint8(x))

        generated += current_bs
        print(f"  [VAE] Geradas: {generated}/{n_samples}")

    samples = torch.cat(outputs, dim=0)[:n_samples]
    print(f"[VAE] Shape final: {samples.shape}")
    return samples
```

In [19]:
```
@torch.no_grad()
def sample_from_gan(generator, n_samples=5000, noise_dim=100, batch_size=256, de
    print(f"[GAN] A gerar {n_samples} imagens...")
    generator.eval()

    outputs = []
    generated = 0

    while generated < n_samples:
        current_bs = min(batch_size, n_samples - generated)
        z = torch.randn(current_bs, noise_dim, 1, 1, device=device)
        x = generator(z)

        outputs.append(to_uint8(x))

        generated += current_bs
        print(f"  [GAN] Geradas: {generated}/{n_samples}")

    samples = torch.cat(outputs, dim=0)[:n_samples]
    print(f"[GAN] Shape final: {samples.shape}")
    return samples
```

In [20]:
```
@torch.no_grad()
def sample_from_diffusion(model, n_samples=5000, img_size=32, channels=3, batch_
    print(f"[Diffusion] A gerar {n_samples} imagens...")
    model.eval()

    outputs = []

    alpha = 1.0 - betas
    alpha_bar = torch.cumprod(alpha, dim=0)

    generated = 0

    while generated < n_samples:
        current_bs = min(batch_size, n_samples - generated)
```

```
        x = torch.randn(current_bs, channels, img_size, img_size, device=device)

        for t in reversed(range(T_STEPS)):
            tt = torch.full((current_bs,), t, device=device, dtype=torch.long)
            pred_noise = model(x, tt)

            a_t = alpha[t]
            ab_t = alpha_bar[t]
            b_t = betas[t]

            if t > 0:
                z = torch.randn_like(x)
            else:
                z = torch.zeros_like(x)

            x = (1 / torch.sqrt(a_t)) * (
                x - ((1 - a_t) / torch.sqrt(1 - ab_t)) * pred_noise
            ) + torch.sqrt(b_t) * z

        outputs.append(to_uint8(x))

        generated += current_bs
        print(f"  [Diffusion] Geradas: {generated}/{n_samples}")

        if torch.cuda.is_available():
            torch.cuda.empty_cache()

    samples = torch.cat(outputs, dim=0)[:n_samples]
    print(f"[Diffusion] Shape final: {samples.shape}")
    return samples
```

In [21]:

```
def show_generated_grid(images_uint8, title, n=36, nrow=6):
    x = images_uint8[:n].float() / 255.0
    grid = make_grid(x, nrow=nrow, padding=2)

    plt.figure(figsize=(8, 8))
    plt.imshow(grid.permute(1, 2, 0).cpu().numpy())
    plt.axis("off")
    plt.title(title)
    plt.show()
```

In [22]:

```
def compute_fid_kid(real_uint8, fake_uint8, kid_subsets=50, kid_subset_size=100,
    """
    Calcula FID e KID em chunks para evitar estouro de memória.
    Espera tensores uint8 no formato [N, C, H, W].
    """
    print("[Metrics] A calcular FID e KID por batches...")

    fid_metric = FrechetInceptionDistance(feature=2048, normalize=False).to(devi
    kid_metric = KernelInceptionDistance(
        feature=2048,
        subsets=kid_subsets,
        subset_size=kid_subset_size,
        normalize=False
    ).to(device)

    n_real = real_uint8.size(0)
    n_fake = fake_uint8.size(0)
```

```
    print(f"[Metrics] Nº imagens reais: {n_real}")
    print(f"[Metrics] Nº imagens falsas: {n_fake}")
    print(f"[Metrics] Batch size métricas: {batch_size}")

    # Atualizar com imagens reais em chunks
    for start in range(0, n_real, batch_size):
        end = min(start + batch_size, n_real)
        real_batch = real_uint8[start:end].to(device)

        fid_metric.update(real_batch, real=True)
        kid_metric.update(real_batch, real=True)

        if device != "cpu" and torch.cuda.is_available():
            torch.cuda.empty_cache()

    # Atualizar com imagens falsas em chunks
    for start in range(0, n_fake, batch_size):
        end = min(start + batch_size, n_fake)
        fake_batch = fake_uint8[start:end].to(device)

        fid_metric.update(fake_batch, real=False)
        kid_metric.update(fake_batch, real=False)

        if device != "cpu" and torch.cuda.is_available():
            torch.cuda.empty_cache()

    fid_value = float(fid_metric.compute().cpu())
    kid_mean, kid_std = kid_metric.compute()

    kid_mean = float(kid_mean.cpu())
    kid_std = float(kid_std.cpu())

    print(f"[Metrics] FID = {fid_value:.4f}")
    print(f"[Metrics] KID mean = {kid_mean:.6f}")
    print(f"[Metrics] KID std  = {kid_std:.6f}")

    return fid_value, kid_mean, kid_std
```

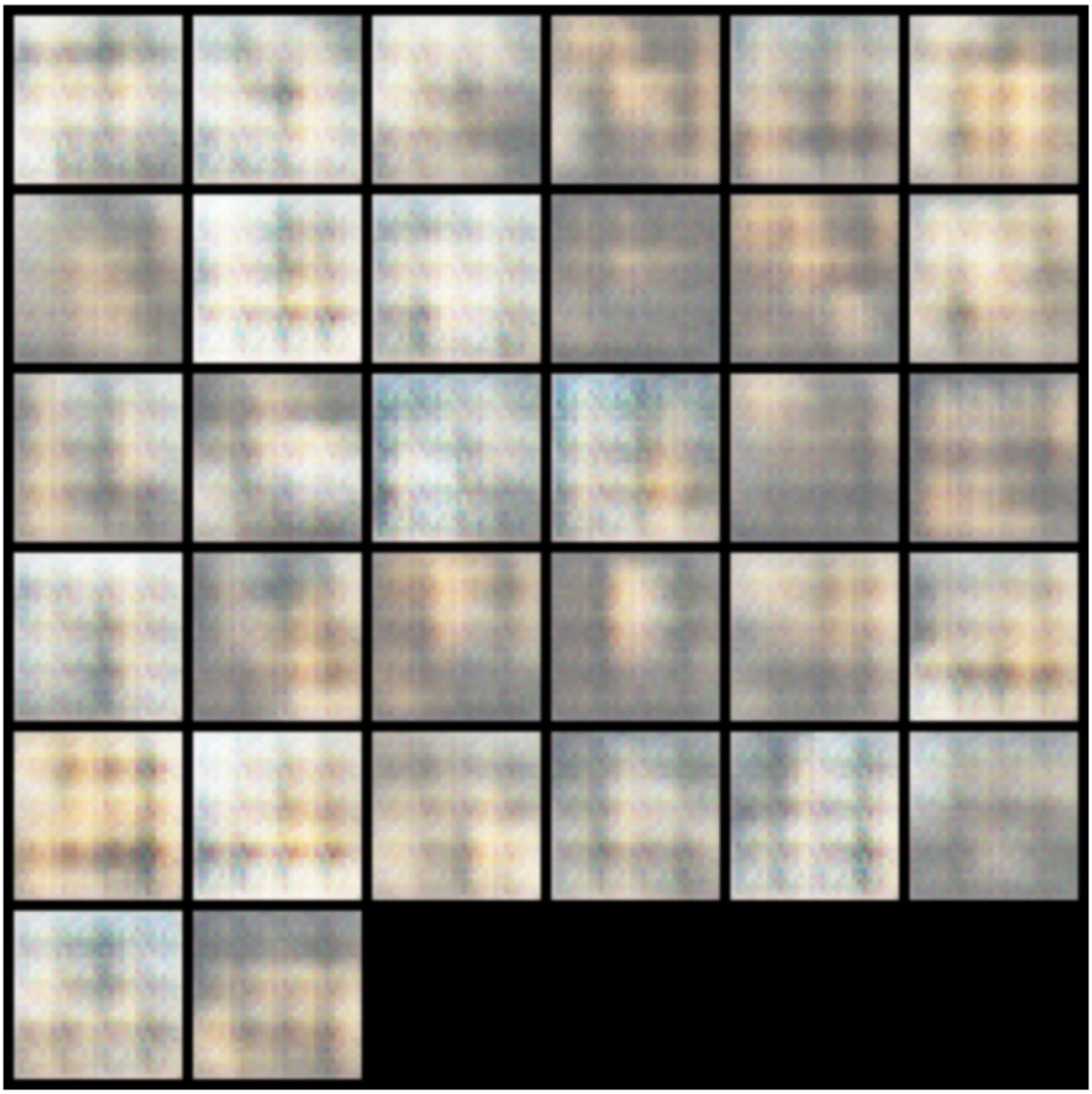
In [23]:

```
real_test = collect_real_images(test_loader, n_samples=32)

vae_test = sample_from_vae(vae_model, n_samples=32, latent_dim=LATENT_DIM, batch
gan_test = sample_from_gan(gan_generator, n_samples=32, noise_dim=NOISE_DIM, bat
diff_test = sample_from_diffusion(diffusion_model, n_samples=16, img_size=32, ch

show_generated_grid(vae_test, "VAE samples")
show_generated_grid(gan_test, "GAN samples")
show_generated_grid(diff_test, "Diffusion samples")
```

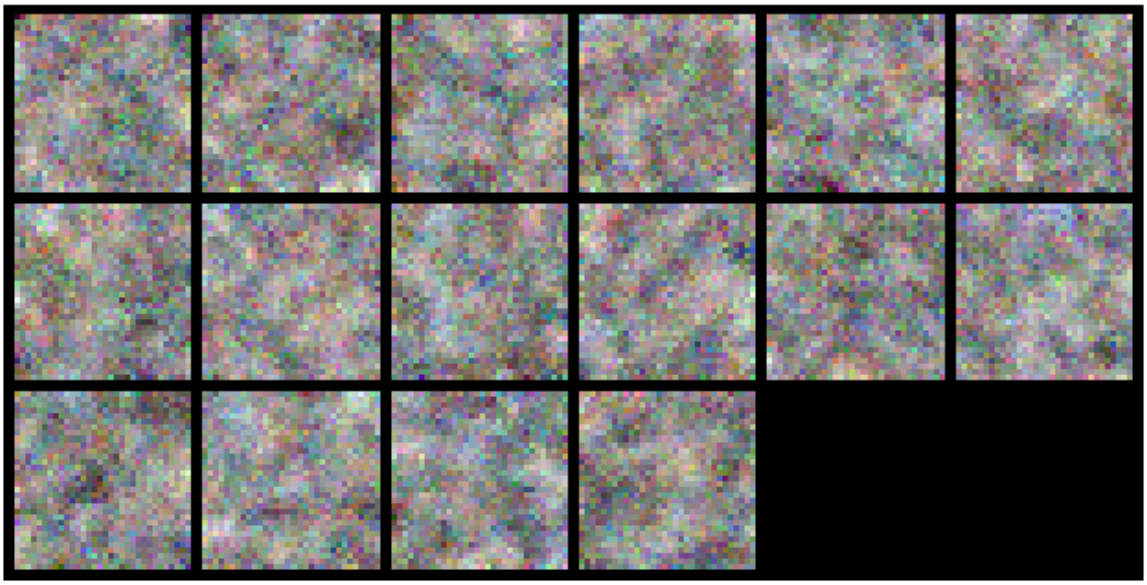
```
[Real] A recolher 32 imagens reais...
  Batch 1: acumuladas = 64
[Real] Shape final: torch.Size([32, 3, 32, 32])
[VAE] A gerar 32 imagens...
  [VAE] Geradas: 32/32
[VAE] Shape final: torch.Size([32, 3, 32, 32])
[GAN] A gerar 32 imagens...
  [GAN] Geradas: 32/32
[GAN] Shape final: torch.Size([32, 3, 32, 32])
[Diffusion] A gerar 16 imagens...
  [Diffusion] Geradas: 8/16
  [Diffusion] Geradas: 16/16
[Diffusion] Shape final: torch.Size([16, 3, 32, 32])
```

VAE samples

GAN samples



Diffusion samples



In [24]:
```
def set_all_seeds(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```python
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)


N_REPETITIONS = 10
N_EVAL_SAMPLES = 5000

results = {
    "VAE": {"fid": [], "kid_mean": [], "kid_std": []},
    "DCGAN": {"fid": [], "kid_mean": [], "kid_std": []},
    "Diffusion": {"fid": [], "kid_mean": [], "kid_std": []},
}

print("=" * 70)
print("INÍCIO DO PROTOCOLO OFICIAL DE AVALIAÇÃO")
print("=" * 70)

real_5000 = collect_real_images(test_loader, n_samples=N_EVAL_SAMPLES)

for repetition in range(N_REPETITIONS):
    current_seed = SEED + repetition

    print("\n" + "#" * 70)
    print(f"Repetição {repetition + 1}/{N_REPETITIONS} | seed = {current_seed}")
    print("#" * 70)

    set_all_seeds(current_seed)

    vae_samples = sample_from_vae(
        vae_model,
        n_samples=N_EVAL_SAMPLES,
        latent_dim=LATENT_DIM,
        batch_size=256,
        device=device
    )

    gan_samples = sample_from_gan(
        gan_generator,
        n_samples=N_EVAL_SAMPLES,
        noise_dim=NOISE_DIM,
        batch_size=256,
        device=device
    )

    diff_samples = sample_from_diffusion(
        diffusion_model,
        n_samples=N_EVAL_SAMPLES,
        img_size=32,
        channels=3,
        batch_size=64,
        device=device
    )

    for model_name, fake_imgs in {
        "VAE": vae_samples,
        "DCGAN": gan_samples,
        "Diffusion": diff_samples
    }.items():
```

```
        print("\n" + "-" * 50)
        print(f"A avaliar: {model_name}")
        print("-" * 50)

        fid_value, kid_mean, kid_std = compute_fid_kid(
            real_5000,
            fake_imgs,
            kid_subsets=50,
            kid_subset_size=100,
            device=device
        )

        results[model_name]["fid"].append(fid_value)
        results[model_name]["kid_mean"].append(kid_mean)
        results[model_name]["kid_std"].append(kid_std)

        if torch.cuda.is_available():
            torch.cuda.empty_cache()

print("\nProtocolo concluído.")
```

```
======================================================================
INÍCIO DO PROTOCOLO OFICIAL DE AVALIAÇÃO
======================================================================
[Real] A recolher 5000 imagens reais...
  Batch 1: acumuladas = 64
  Batch 2: acumuladas = 128
  Batch 3: acumuladas = 192
  Batch 4: acumuladas = 256
  Batch 5: acumuladas = 320
  Batch 6: acumuladas = 384
  Batch 7: acumuladas = 448
  Batch 8: acumuladas = 512
  Batch 9: acumuladas = 576
  Batch 10: acumuladas = 640
  Batch 11: acumuladas = 704
  Batch 12: acumuladas = 768
  Batch 13: acumuladas = 832
  Batch 14: acumuladas = 896
  Batch 15: acumuladas = 960
  Batch 16: acumuladas = 1024
  Batch 17: acumuladas = 1088
  Batch 18: acumuladas = 1152
  Batch 19: acumuladas = 1216
  Batch 20: acumuladas = 1280
  Batch 21: acumuladas = 1344
  Batch 22: acumuladas = 1408
  Batch 23: acumuladas = 1472
  Batch 24: acumuladas = 1536
  Batch 25: acumuladas = 1600
  Batch 26: acumuladas = 1664
  Batch 27: acumuladas = 1728
  Batch 28: acumuladas = 1792
  Batch 29: acumuladas = 1856
  Batch 30: acumuladas = 1920
  Batch 31: acumuladas = 1984
  Batch 32: acumuladas = 2048
  Batch 33: acumuladas = 2112
  Batch 34: acumuladas = 2176
  Batch 35: acumuladas = 2240
  Batch 36: acumuladas = 2304
  Batch 37: acumuladas = 2368
  Batch 38: acumuladas = 2432
  Batch 39: acumuladas = 2496
  Batch 40: acumuladas = 2560
  Batch 41: acumuladas = 2624
  Batch 42: acumuladas = 2688
  Batch 43: acumuladas = 2752
  Batch 44: acumuladas = 2816
  Batch 45: acumuladas = 2880
  Batch 46: acumuladas = 2944
  Batch 47: acumuladas = 3008
  Batch 48: acumuladas = 3072
  Batch 49: acumuladas = 3136
  Batch 50: acumuladas = 3200
  Batch 51: acumuladas = 3264
  Batch 52: acumuladas = 3328
  Batch 53: acumuladas = 3392
  Batch 54: acumuladas = 3456
  Batch 55: acumuladas = 3520
  Batch 56: acumuladas = 3584
```

```
  Batch 57: acumuladas = 3648
  Batch 58: acumuladas = 3712
  Batch 59: acumuladas = 3776
  Batch 60: acumuladas = 3840
  Batch 61: acumuladas = 3904
  Batch 62: acumuladas = 3968
  Batch 63: acumuladas = 4032
  Batch 64: acumuladas = 4096
  Batch 65: acumuladas = 4160
  Batch 66: acumuladas = 4224
  Batch 67: acumuladas = 4288
  Batch 68: acumuladas = 4352
  Batch 69: acumuladas = 4416
  Batch 70: acumuladas = 4480
  Batch 71: acumuladas = 4544
  Batch 72: acumuladas = 4608
  Batch 73: acumuladas = 4672
  Batch 74: acumuladas = 4736
  Batch 75: acumuladas = 4800
  Batch 76: acumuladas = 4864
  Batch 77: acumuladas = 4928
  Batch 78: acumuladas = 4992
  Batch 79: acumuladas = 5056
[Real] Shape final: torch.Size([5000, 3, 32, 32])

######################################################################
Repetição 1/10 | seed = 42
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
```

```
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
```

```
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
```

```
c:\Users\joaoa\Desktop\Lara\.venv\Lib\site-packages\torchmetrics\utilities\print
s.py:43: UserWarning: Metric `Kernel Inception Distance` will save all extracted
features in buffer. For large datasets this may lead to large memory footprint.
  warnings.warn(*args, **kwargs)
```

```
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.3026
[Metrics] KID mean = 0.393136
[Metrics] KID std  = 0.013712

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 372.0614
[Metrics] KID mean = 0.389392
[Metrics] KID std  = 0.011712

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 386.4429
[Metrics] KID mean = 0.431425
[Metrics] KID std  = 0.013377

######################################################################
Repetição 2/10 | seed = 43
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
```

```
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
```

```
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 353.3414
[Metrics] KID mean = 0.387293
[Metrics] KID std  = 0.014832

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.2866
[Metrics] KID mean = 0.386191
[Metrics] KID std  = 0.012923
```

```
--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 386.5874
[Metrics] KID mean = 0.432892
[Metrics] KID std  = 0.012941

######################################################################
Repetição 3/10 | seed = 44
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
```

```
[Diffusion] Geradas: 128/5000
[Diffusion] Geradas: 192/5000
[Diffusion] Geradas: 256/5000
[Diffusion] Geradas: 320/5000
[Diffusion] Geradas: 384/5000
[Diffusion] Geradas: 448/5000
[Diffusion] Geradas: 512/5000
[Diffusion] Geradas: 576/5000
[Diffusion] Geradas: 640/5000
[Diffusion] Geradas: 704/5000
[Diffusion] Geradas: 768/5000
[Diffusion] Geradas: 832/5000
[Diffusion] Geradas: 896/5000
[Diffusion] Geradas: 960/5000
[Diffusion] Geradas: 1024/5000
[Diffusion] Geradas: 1088/5000
[Diffusion] Geradas: 1152/5000
[Diffusion] Geradas: 1216/5000
[Diffusion] Geradas: 1280/5000
[Diffusion] Geradas: 1344/5000
[Diffusion] Geradas: 1408/5000
[Diffusion] Geradas: 1472/5000
[Diffusion] Geradas: 1536/5000
[Diffusion] Geradas: 1600/5000
[Diffusion] Geradas: 1664/5000
[Diffusion] Geradas: 1728/5000
[Diffusion] Geradas: 1792/5000
[Diffusion] Geradas: 1856/5000
[Diffusion] Geradas: 1920/5000
[Diffusion] Geradas: 1984/5000
[Diffusion] Geradas: 2048/5000
[Diffusion] Geradas: 2112/5000
[Diffusion] Geradas: 2176/5000
[Diffusion] Geradas: 2240/5000
[Diffusion] Geradas: 2304/5000
[Diffusion] Geradas: 2368/5000
[Diffusion] Geradas: 2432/5000
[Diffusion] Geradas: 2496/5000
[Diffusion] Geradas: 2560/5000
[Diffusion] Geradas: 2624/5000
[Diffusion] Geradas: 2688/5000
[Diffusion] Geradas: 2752/5000
[Diffusion] Geradas: 2816/5000
[Diffusion] Geradas: 2880/5000
[Diffusion] Geradas: 2944/5000
[Diffusion] Geradas: 3008/5000
[Diffusion] Geradas: 3072/5000
[Diffusion] Geradas: 3136/5000
[Diffusion] Geradas: 3200/5000
[Diffusion] Geradas: 3264/5000
[Diffusion] Geradas: 3328/5000
[Diffusion] Geradas: 3392/5000
[Diffusion] Geradas: 3456/5000
[Diffusion] Geradas: 3520/5000
[Diffusion] Geradas: 3584/5000
[Diffusion] Geradas: 3648/5000
[Diffusion] Geradas: 3712/5000
[Diffusion] Geradas: 3776/5000
[Diffusion] Geradas: 3840/5000
[Diffusion] Geradas: 3904/5000
```

```
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.5001
[Metrics] KID mean = 0.391238
[Metrics] KID std  = 0.015937

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.9324
[Metrics] KID mean = 0.390296
[Metrics] KID std  = 0.011827

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 385.6418
[Metrics] KID mean = 0.433241
[Metrics] KID std  = 0.012644

######################################################################
Repetição 4/10 | seed = 45
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
```

```
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
```

```
[Diffusion] Geradas: 1280/5000
[Diffusion] Geradas: 1344/5000
[Diffusion] Geradas: 1408/5000
[Diffusion] Geradas: 1472/5000
[Diffusion] Geradas: 1536/5000
[Diffusion] Geradas: 1600/5000
[Diffusion] Geradas: 1664/5000
[Diffusion] Geradas: 1728/5000
[Diffusion] Geradas: 1792/5000
[Diffusion] Geradas: 1856/5000
[Diffusion] Geradas: 1920/5000
[Diffusion] Geradas: 1984/5000
[Diffusion] Geradas: 2048/5000
[Diffusion] Geradas: 2112/5000
[Diffusion] Geradas: 2176/5000
[Diffusion] Geradas: 2240/5000
[Diffusion] Geradas: 2304/5000
[Diffusion] Geradas: 2368/5000
[Diffusion] Geradas: 2432/5000
[Diffusion] Geradas: 2496/5000
[Diffusion] Geradas: 2560/5000
[Diffusion] Geradas: 2624/5000
[Diffusion] Geradas: 2688/5000
[Diffusion] Geradas: 2752/5000
[Diffusion] Geradas: 2816/5000
[Diffusion] Geradas: 2880/5000
[Diffusion] Geradas: 2944/5000
[Diffusion] Geradas: 3008/5000
[Diffusion] Geradas: 3072/5000
[Diffusion] Geradas: 3136/5000
[Diffusion] Geradas: 3200/5000
[Diffusion] Geradas: 3264/5000
[Diffusion] Geradas: 3328/5000
[Diffusion] Geradas: 3392/5000
[Diffusion] Geradas: 3456/5000
[Diffusion] Geradas: 3520/5000
[Diffusion] Geradas: 3584/5000
[Diffusion] Geradas: 3648/5000
[Diffusion] Geradas: 3712/5000
[Diffusion] Geradas: 3776/5000
[Diffusion] Geradas: 3840/5000
[Diffusion] Geradas: 3904/5000
[Diffusion] Geradas: 3968/5000
[Diffusion] Geradas: 4032/5000
[Diffusion] Geradas: 4096/5000
[Diffusion] Geradas: 4160/5000
[Diffusion] Geradas: 4224/5000
[Diffusion] Geradas: 4288/5000
[Diffusion] Geradas: 4352/5000
[Diffusion] Geradas: 4416/5000
[Diffusion] Geradas: 4480/5000
[Diffusion] Geradas: 4544/5000
[Diffusion] Geradas: 4608/5000
[Diffusion] Geradas: 4672/5000
[Diffusion] Geradas: 4736/5000
[Diffusion] Geradas: 4800/5000
[Diffusion] Geradas: 4864/5000
[Diffusion] Geradas: 4928/5000
[Diffusion] Geradas: 4992/5000
[Diffusion] Geradas: 5000/5000
```

```
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.3395
[Metrics] KID mean = 0.386100
[Metrics] KID std  = 0.014360

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.2291
[Metrics] KID mean = 0.388869
[Metrics] KID std  = 0.013498

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 385.9875
[Metrics] KID mean = 0.430874
[Metrics] KID std  = 0.013627

######################################################################
Repetição 5/10 | seed = 46
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
```

```
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
```

```
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.2878
[Metrics] KID mean = 0.388493
[Metrics] KID std  = 0.016281

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
```

```
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.9768
[Metrics] KID mean = 0.390176
[Metrics] KID std  = 0.013772

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 385.7093
[Metrics] KID mean = 0.432747
[Metrics] KID std  = 0.014911

######################################################################
Repetição 6/10 | seed = 47
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
```

```
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
```

```
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 353.3175
[Metrics] KID mean = 0.389365
[Metrics] KID std  = 0.015613

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.8937
[Metrics] KID mean = 0.388118
[Metrics] KID std  = 0.010335

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 386.3520
[Metrics] KID mean = 0.434190
[Metrics] KID std  = 0.013391

#######################################################################
```

```
Repetição 7/10 | seed = 48
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
```

```
[Diffusion] Geradas: 896/5000
[Diffusion] Geradas: 960/5000
[Diffusion] Geradas: 1024/5000
[Diffusion] Geradas: 1088/5000
[Diffusion] Geradas: 1152/5000
[Diffusion] Geradas: 1216/5000
[Diffusion] Geradas: 1280/5000
[Diffusion] Geradas: 1344/5000
[Diffusion] Geradas: 1408/5000
[Diffusion] Geradas: 1472/5000
[Diffusion] Geradas: 1536/5000
[Diffusion] Geradas: 1600/5000
[Diffusion] Geradas: 1664/5000
[Diffusion] Geradas: 1728/5000
[Diffusion] Geradas: 1792/5000
[Diffusion] Geradas: 1856/5000
[Diffusion] Geradas: 1920/5000
[Diffusion] Geradas: 1984/5000
[Diffusion] Geradas: 2048/5000
[Diffusion] Geradas: 2112/5000
[Diffusion] Geradas: 2176/5000
[Diffusion] Geradas: 2240/5000
[Diffusion] Geradas: 2304/5000
[Diffusion] Geradas: 2368/5000
[Diffusion] Geradas: 2432/5000
[Diffusion] Geradas: 2496/5000
[Diffusion] Geradas: 2560/5000
[Diffusion] Geradas: 2624/5000
[Diffusion] Geradas: 2688/5000
[Diffusion] Geradas: 2752/5000
[Diffusion] Geradas: 2816/5000
[Diffusion] Geradas: 2880/5000
[Diffusion] Geradas: 2944/5000
[Diffusion] Geradas: 3008/5000
[Diffusion] Geradas: 3072/5000
[Diffusion] Geradas: 3136/5000
[Diffusion] Geradas: 3200/5000
[Diffusion] Geradas: 3264/5000
[Diffusion] Geradas: 3328/5000
[Diffusion] Geradas: 3392/5000
[Diffusion] Geradas: 3456/5000
[Diffusion] Geradas: 3520/5000
[Diffusion] Geradas: 3584/5000
[Diffusion] Geradas: 3648/5000
[Diffusion] Geradas: 3712/5000
[Diffusion] Geradas: 3776/5000
[Diffusion] Geradas: 3840/5000
[Diffusion] Geradas: 3904/5000
[Diffusion] Geradas: 3968/5000
[Diffusion] Geradas: 4032/5000
[Diffusion] Geradas: 4096/5000
[Diffusion] Geradas: 4160/5000
[Diffusion] Geradas: 4224/5000
[Diffusion] Geradas: 4288/5000
[Diffusion] Geradas: 4352/5000
[Diffusion] Geradas: 4416/5000
[Diffusion] Geradas: 4480/5000
[Diffusion] Geradas: 4544/5000
[Diffusion] Geradas: 4608/5000
[Diffusion] Geradas: 4672/5000
```

```
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.9855
[Metrics] KID mean = 0.394122
[Metrics] KID std  = 0.016682

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 371.4189
[Metrics] KID mean = 0.387235
[Metrics] KID std  = 0.010249

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 386.3977
[Metrics] KID mean = 0.435459
[Metrics] KID std  = 0.014420

######################################################################
Repetição 8/10 | seed = 49
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
```

```
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
```

```
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 354.1381
[Metrics] KID mean = 0.390002
[Metrics] KID std  = 0.014919
```

```
--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 372.5318
[Metrics] KID mean = 0.390522
[Metrics] KID std  = 0.011324

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 386.1278
[Metrics] KID mean = 0.434494
[Metrics] KID std  = 0.014198

######################################################################
Repetição 9/10 | seed = 50
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
```

```
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
  [Diffusion] Geradas: 512/5000
  [Diffusion] Geradas: 576/5000
  [Diffusion] Geradas: 640/5000
  [Diffusion] Geradas: 704/5000
  [Diffusion] Geradas: 768/5000
  [Diffusion] Geradas: 832/5000
  [Diffusion] Geradas: 896/5000
  [Diffusion] Geradas: 960/5000
  [Diffusion] Geradas: 1024/5000
  [Diffusion] Geradas: 1088/5000
  [Diffusion] Geradas: 1152/5000
  [Diffusion] Geradas: 1216/5000
  [Diffusion] Geradas: 1280/5000
  [Diffusion] Geradas: 1344/5000
  [Diffusion] Geradas: 1408/5000
  [Diffusion] Geradas: 1472/5000
  [Diffusion] Geradas: 1536/5000
  [Diffusion] Geradas: 1600/5000
  [Diffusion] Geradas: 1664/5000
  [Diffusion] Geradas: 1728/5000
  [Diffusion] Geradas: 1792/5000
  [Diffusion] Geradas: 1856/5000
  [Diffusion] Geradas: 1920/5000
  [Diffusion] Geradas: 1984/5000
  [Diffusion] Geradas: 2048/5000
  [Diffusion] Geradas: 2112/5000
  [Diffusion] Geradas: 2176/5000
  [Diffusion] Geradas: 2240/5000
  [Diffusion] Geradas: 2304/5000
  [Diffusion] Geradas: 2368/5000
  [Diffusion] Geradas: 2432/5000
  [Diffusion] Geradas: 2496/5000
  [Diffusion] Geradas: 2560/5000
  [Diffusion] Geradas: 2624/5000
  [Diffusion] Geradas: 2688/5000
  [Diffusion] Geradas: 2752/5000
  [Diffusion] Geradas: 2816/5000
  [Diffusion] Geradas: 2880/5000
  [Diffusion] Geradas: 2944/5000
  [Diffusion] Geradas: 3008/5000
  [Diffusion] Geradas: 3072/5000
  [Diffusion] Geradas: 3136/5000
```

```
  [Diffusion] Geradas: 3200/5000
  [Diffusion] Geradas: 3264/5000
  [Diffusion] Geradas: 3328/5000
  [Diffusion] Geradas: 3392/5000
  [Diffusion] Geradas: 3456/5000
  [Diffusion] Geradas: 3520/5000
  [Diffusion] Geradas: 3584/5000
  [Diffusion] Geradas: 3648/5000
  [Diffusion] Geradas: 3712/5000
  [Diffusion] Geradas: 3776/5000
  [Diffusion] Geradas: 3840/5000
  [Diffusion] Geradas: 3904/5000
  [Diffusion] Geradas: 3968/5000
  [Diffusion] Geradas: 4032/5000
  [Diffusion] Geradas: 4096/5000
  [Diffusion] Geradas: 4160/5000
  [Diffusion] Geradas: 4224/5000
  [Diffusion] Geradas: 4288/5000
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 353.6308
[Metrics] KID mean = 0.389100
[Metrics] KID std  = 0.012814

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 370.2067
[Metrics] KID mean = 0.392335
[Metrics] KID std  = 0.013729

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
```

```
[Metrics] Batch size métricas: 64
[Metrics] FID = 385.2330
[Metrics] KID mean = 0.434241
[Metrics] KID std  = 0.014746

######################################################################
Repetição 10/10 | seed = 51
######################################################################
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[GAN] A gerar 5000 imagens...
  [GAN] Geradas: 256/5000
  [GAN] Geradas: 512/5000
  [GAN] Geradas: 768/5000
  [GAN] Geradas: 1024/5000
  [GAN] Geradas: 1280/5000
  [GAN] Geradas: 1536/5000
  [GAN] Geradas: 1792/5000
  [GAN] Geradas: 2048/5000
  [GAN] Geradas: 2304/5000
  [GAN] Geradas: 2560/5000
  [GAN] Geradas: 2816/5000
  [GAN] Geradas: 3072/5000
  [GAN] Geradas: 3328/5000
  [GAN] Geradas: 3584/5000
  [GAN] Geradas: 3840/5000
  [GAN] Geradas: 4096/5000
  [GAN] Geradas: 4352/5000
  [GAN] Geradas: 4608/5000
  [GAN] Geradas: 4864/5000
  [GAN] Geradas: 5000/5000
[GAN] Shape final: torch.Size([5000, 3, 32, 32])
[Diffusion] A gerar 5000 imagens...
  [Diffusion] Geradas: 64/5000
  [Diffusion] Geradas: 128/5000
  [Diffusion] Geradas: 192/5000
  [Diffusion] Geradas: 256/5000
  [Diffusion] Geradas: 320/5000
  [Diffusion] Geradas: 384/5000
  [Diffusion] Geradas: 448/5000
```

```
[Diffusion] Geradas: 512/5000
[Diffusion] Geradas: 576/5000
[Diffusion] Geradas: 640/5000
[Diffusion] Geradas: 704/5000
[Diffusion] Geradas: 768/5000
[Diffusion] Geradas: 832/5000
[Diffusion] Geradas: 896/5000
[Diffusion] Geradas: 960/5000
[Diffusion] Geradas: 1024/5000
[Diffusion] Geradas: 1088/5000
[Diffusion] Geradas: 1152/5000
[Diffusion] Geradas: 1216/5000
[Diffusion] Geradas: 1280/5000
[Diffusion] Geradas: 1344/5000
[Diffusion] Geradas: 1408/5000
[Diffusion] Geradas: 1472/5000
[Diffusion] Geradas: 1536/5000
[Diffusion] Geradas: 1600/5000
[Diffusion] Geradas: 1664/5000
[Diffusion] Geradas: 1728/5000
[Diffusion] Geradas: 1792/5000
[Diffusion] Geradas: 1856/5000
[Diffusion] Geradas: 1920/5000
[Diffusion] Geradas: 1984/5000
[Diffusion] Geradas: 2048/5000
[Diffusion] Geradas: 2112/5000
[Diffusion] Geradas: 2176/5000
[Diffusion] Geradas: 2240/5000
[Diffusion] Geradas: 2304/5000
[Diffusion] Geradas: 2368/5000
[Diffusion] Geradas: 2432/5000
[Diffusion] Geradas: 2496/5000
[Diffusion] Geradas: 2560/5000
[Diffusion] Geradas: 2624/5000
[Diffusion] Geradas: 2688/5000
[Diffusion] Geradas: 2752/5000
[Diffusion] Geradas: 2816/5000
[Diffusion] Geradas: 2880/5000
[Diffusion] Geradas: 2944/5000
[Diffusion] Geradas: 3008/5000
[Diffusion] Geradas: 3072/5000
[Diffusion] Geradas: 3136/5000
[Diffusion] Geradas: 3200/5000
[Diffusion] Geradas: 3264/5000
[Diffusion] Geradas: 3328/5000
[Diffusion] Geradas: 3392/5000
[Diffusion] Geradas: 3456/5000
[Diffusion] Geradas: 3520/5000
[Diffusion] Geradas: 3584/5000
[Diffusion] Geradas: 3648/5000
[Diffusion] Geradas: 3712/5000
[Diffusion] Geradas: 3776/5000
[Diffusion] Geradas: 3840/5000
[Diffusion] Geradas: 3904/5000
[Diffusion] Geradas: 3968/5000
[Diffusion] Geradas: 4032/5000
[Diffusion] Geradas: 4096/5000
[Diffusion] Geradas: 4160/5000
[Diffusion] Geradas: 4224/5000
[Diffusion] Geradas: 4288/5000
```

```
  [Diffusion] Geradas: 4352/5000
  [Diffusion] Geradas: 4416/5000
  [Diffusion] Geradas: 4480/5000
  [Diffusion] Geradas: 4544/5000
  [Diffusion] Geradas: 4608/5000
  [Diffusion] Geradas: 4672/5000
  [Diffusion] Geradas: 4736/5000
  [Diffusion] Geradas: 4800/5000
  [Diffusion] Geradas: 4864/5000
  [Diffusion] Geradas: 4928/5000
  [Diffusion] Geradas: 4992/5000
  [Diffusion] Geradas: 5000/5000
[Diffusion] Shape final: torch.Size([5000, 3, 32, 32])

--------------------------------------------------
A avaliar: VAE
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 353.6510
[Metrics] KID mean = 0.388858
[Metrics] KID std  = 0.015998

--------------------------------------------------
A avaliar: DCGAN
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 372.1819
[Metrics] KID mean = 0.387163
[Metrics] KID std  = 0.012846

--------------------------------------------------
A avaliar: Diffusion
--------------------------------------------------
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 64
[Metrics] FID = 385.4014
[Metrics] KID mean = 0.431873
[Metrics] KID std  = 0.011042

Protocolo concluído.
```

In [25]:

```python
print("\n" + "=" * 70)
print(f"RESULTADOS FINAIS (ArtBench-10, n={N_EVAL_SAMPLES})")
print("=" * 70)

for model_name, metrics in results.items():
    fid_mean = np.mean(metrics["fid"])
    fid_std = np.std(metrics["fid"])

    kid_mean_mean = np.mean(metrics["kid_mean"])
    kid_mean_std = np.std(metrics["kid_mean"])
```

```
    print(f"\n{model_name}")
    print(f"FID: {fid_mean:.4f} ± {fid_std:.4f}")
    print(f"KID: {kid_mean_mean:.6f} ± {kid_mean_std:.6f}")
```

```
======================================================================
RESULTADOS FINAIS (ArtBench-10, n=5000)
======================================================================

VAE
FID: 354.0494 ± 0.5161
KID: 0.389771 ± 0.002349

DCGAN
FID: 371.6719 ± 0.6258
KID: 0.389030 ± 0.001784

Diffusion
FID: 385.9881 ± 0.4470
KID: 0.433144 ± 0.001394
```

In [26]:

```
print("\n" + "=" * 70)
print("SELEÇÃO DO MELHOR MODELO")
print("=" * 70)

# calcular média FID
avg_fid = {
    model: np.mean(metrics["fid"])
    for model, metrics in results.items()
}

for model, value in avg_fid.items():
    print(f"{model}: FID médio = {value:.4f}")

best_model_type = min(avg_fid, key=avg_fid.get)

print(f"\nMelhor modelo selecionado: {best_model_type}")
```

```
======================================================================
SELEÇÃO DO MELHOR MODELO
======================================================================
VAE: FID médio = 354.0494
DCGAN: FID médio = 371.6719
Diffusion: FID médio = 385.9881

Melhor modelo selecionado: VAE
```

In [27]:

```
print("\n" + "=" * 70)
print("INICIALIZAÇÃO DO MODELO FINAL")
print("=" * 70)

if best_model_type == "VAE":
    best_model = SimpleVAE(latent_dim=LATENT_DIM).to(device)
    best_optimizer = torch.optim.Adam(best_model.parameters(), lr=LR_VAE)

elif best_model_type == "DCGAN":
    best_model = DCGANGenerator(noise_dim=NOISE_DIM).to(device)
    best_discriminator = DCGANDiscriminator().to(device)

elif best_model_type == "Diffusion":
```

```
    best_model = TinyUNet().to(device)
    best_optimizer = torch.optim.Adam(best_model.parameters(), lr=LR_DIFF)
```

```
======================================================================
INICIALIZAÇÃO DO MODELO FINAL
======================================================================
```

In [28]:
```
print("\n" + "=" * 70)
print("TREINO FINAL NO DATASET COMPLETO")
print("=" * 70)

FINAL_EPOCHS = 10

if best_model_type == "VAE":
    train_vae(best_model, train_loader, best_optimizer, FINAL_EPOCHS, device)

elif best_model_type == "DCGAN":
    train_gan(best_model, best_discriminator, train_loader,
              gan_g_optimizer, gan_d_optimizer,
              gan_criterion, FINAL_EPOCHS, device)

elif best_model_type == "Diffusion":
    train_diffusion(best_model, train_loader,
                    best_optimizer, FINAL_EPOCHS,
                    device, betas, alphas_cumprod)
```

```
======================================================================
TREINO FINAL NO DATASET COMPLETO
======================================================================
============================================================
Treino do VAE
============================================================
[VAE] Epoch 1/10 | loss = 91.984389
[VAE] Epoch 2/10 | loss = 72.155569
[VAE] Epoch 3/10 | loss = 70.629117
[VAE] Epoch 4/10 | loss = 70.036719
[VAE] Epoch 5/10 | loss = 69.576374
[VAE] Epoch 6/10 | loss = 69.216976
[VAE] Epoch 7/10 | loss = 69.036815
[VAE] Epoch 8/10 | loss = 68.899121
[VAE] Epoch 9/10 | loss = 68.652212
[VAE] Epoch 10/10 | loss = 68.519732
```

In [29]:
```
print("\n" + "=" * 70)
print("AVALIAÇÃO FINAL (MODELO TREINADO NO DATASET COMPLETO)")
print("=" * 70)

real_5000 = collect_real_images(test_loader, n_samples=5000)

if best_model_type == "VAE":
    final_samples = sample_from_vae(best_model, 5000, LATENT_DIM, device=device)

elif best_model_type == "DCGAN":
    final_samples = sample_from_gan(best_model, 5000, NOISE_DIM, device=device)

elif best_model_type == "Diffusion":
    final_samples = sample_from_diffusion(best_model, 5000, device=device)

fid, kid_mean, kid_std = compute_fid_kid(
    real_5000,
    final_samples,
```

```
    device=device,
    batch_size=32
)

print("\n" + "=" * 70)
print("RESULTADO FINAL (FULL DATASET)")
print("=" * 70)
print(f"Modelo: {best_model_type}")
print(f"FID: {fid:.4f}")
print(f"KID: {kid_mean:.6f}")
```

```
======================================================================
AVALIAÇÃO FINAL (MODELO TREINADO NO DATASET COMPLETO)
======================================================================
[Real] A recolher 5000 imagens reais...
  Batch 1: acumuladas = 64
  Batch 2: acumuladas = 128
  Batch 3: acumuladas = 192
  Batch 4: acumuladas = 256
  Batch 5: acumuladas = 320
  Batch 6: acumuladas = 384
  Batch 7: acumuladas = 448
  Batch 8: acumuladas = 512
  Batch 9: acumuladas = 576
  Batch 10: acumuladas = 640
  Batch 11: acumuladas = 704
  Batch 12: acumuladas = 768
  Batch 13: acumuladas = 832
  Batch 14: acumuladas = 896
  Batch 15: acumuladas = 960
  Batch 16: acumuladas = 1024
  Batch 17: acumuladas = 1088
  Batch 18: acumuladas = 1152
  Batch 19: acumuladas = 1216
  Batch 20: acumuladas = 1280
  Batch 21: acumuladas = 1344
  Batch 22: acumuladas = 1408
  Batch 23: acumuladas = 1472
  Batch 24: acumuladas = 1536
  Batch 25: acumuladas = 1600
  Batch 26: acumuladas = 1664
  Batch 27: acumuladas = 1728
  Batch 28: acumuladas = 1792
  Batch 29: acumuladas = 1856
  Batch 30: acumuladas = 1920
  Batch 31: acumuladas = 1984
  Batch 32: acumuladas = 2048
  Batch 33: acumuladas = 2112
  Batch 34: acumuladas = 2176
  Batch 35: acumuladas = 2240
  Batch 36: acumuladas = 2304
  Batch 37: acumuladas = 2368
  Batch 38: acumuladas = 2432
  Batch 39: acumuladas = 2496
  Batch 40: acumuladas = 2560
  Batch 41: acumuladas = 2624
  Batch 42: acumuladas = 2688
  Batch 43: acumuladas = 2752
  Batch 44: acumuladas = 2816
  Batch 45: acumuladas = 2880
  Batch 46: acumuladas = 2944
  Batch 47: acumuladas = 3008
  Batch 48: acumuladas = 3072
  Batch 49: acumuladas = 3136
  Batch 50: acumuladas = 3200
  Batch 51: acumuladas = 3264
  Batch 52: acumuladas = 3328
  Batch 53: acumuladas = 3392
  Batch 54: acumuladas = 3456
  Batch 55: acumuladas = 3520
  Batch 56: acumuladas = 3584
```

```
  Batch 57: acumuladas = 3648
  Batch 58: acumuladas = 3712
  Batch 59: acumuladas = 3776
  Batch 60: acumuladas = 3840
  Batch 61: acumuladas = 3904
  Batch 62: acumuladas = 3968
  Batch 63: acumuladas = 4032
  Batch 64: acumuladas = 4096
  Batch 65: acumuladas = 4160
  Batch 66: acumuladas = 4224
  Batch 67: acumuladas = 4288
  Batch 68: acumuladas = 4352
  Batch 69: acumuladas = 4416
  Batch 70: acumuladas = 4480
  Batch 71: acumuladas = 4544
  Batch 72: acumuladas = 4608
  Batch 73: acumuladas = 4672
  Batch 74: acumuladas = 4736
  Batch 75: acumuladas = 4800
  Batch 76: acumuladas = 4864
  Batch 77: acumuladas = 4928
  Batch 78: acumuladas = 4992
  Batch 79: acumuladas = 5056
[Real] Shape final: torch.Size([5000, 3, 32, 32])
[VAE] A gerar 5000 imagens...
  [VAE] Geradas: 256/5000
  [VAE] Geradas: 512/5000
  [VAE] Geradas: 768/5000
  [VAE] Geradas: 1024/5000
  [VAE] Geradas: 1280/5000
  [VAE] Geradas: 1536/5000
  [VAE] Geradas: 1792/5000
  [VAE] Geradas: 2048/5000
  [VAE] Geradas: 2304/5000
  [VAE] Geradas: 2560/5000
  [VAE] Geradas: 2816/5000
  [VAE] Geradas: 3072/5000
  [VAE] Geradas: 3328/5000
  [VAE] Geradas: 3584/5000
  [VAE] Geradas: 3840/5000
  [VAE] Geradas: 4096/5000
  [VAE] Geradas: 4352/5000
  [VAE] Geradas: 4608/5000
  [VAE] Geradas: 4864/5000
  [VAE] Geradas: 5000/5000
[VAE] Shape final: torch.Size([5000, 3, 32, 32])
[Metrics] A calcular FID e KID por batches...
[Metrics] Nº imagens reais: 5000
[Metrics] Nº imagens falsas: 5000
[Metrics] Batch size métricas: 32
[Metrics] FID = 238.7489
[Metrics] KID mean = 0.220324
[Metrics] KID std  = 0.009776

======================================================================
RESULTADO FINAL (FULL DATASET)
======================================================================
Modelo: VAE
FID: 238.7489
KID: 0.220324
```

In [30]:
```
final_results = {}

for model_name, metrics in results.items():
    final_results[model_name] = {
        "fid_mean": float(np.mean(metrics["fid"])),
        "fid_std": float(np.std(metrics["fid"])),
        "kid_mean": float(np.mean(metrics["kid_mean"])),
        "kid_std": float(np.std(metrics["kid_mean"]))
    }

print(final_results)
```

```
{'VAE': {'fid_mean': 354.04942932128904, 'fid_std': 0.5160555355298798, 'kid_mea
n': 0.3897708237171173, 'kid_std': 0.0023486692755950345}, 'DCGAN': {'fid_mean':
371.67193603515625, 'fid_std': 0.6258410775830484, 'kid_mean': 0.3890298515558242
6, 'kid_std': 0.0017840387600779556}, 'Diffusion': {'fid_mean': 385.988079833984
4, 'fid_std': 0.44696835175554955, 'kid_mean': 0.4331435769796371, 'kid_std': 0.0
013940124349001279}}
```